

Projeto: Sistema de Gestão Escolar (JDBC + DTOs + SQL)

1. Script SQL para criação das tabelas

```
-- Tabela Aluno
CREATE TABLE aluno (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  nome VARCHAR(100) NOT NULL,
  email VARCHAR(100) NOT NULL UNIQUE,
  matricula VARCHAR(20) NOT NULL,
  data_nascimento DATE NOT NULL
);

-- Tabela Professor
CREATE TABLE professor (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  nome VARCHAR(100) NOT NULL,
  email VARCHAR(100) NOT NULL UNIQUE,
  disciplina VARCHAR(50) NOT NULL
);

-- Tabela Curso
CREATE TABLE curso (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  nome VARCHAR(100) NOT NULL,
  codigo VARCHAR(20) NOT NULL UNIQUE
);

-- Tabela Turma
CREATE TABLE turma (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  nome VARCHAR(50) NOT NULL,
  curso_id BIGINT NOT NULL,
  professor_id BIGINT NOT NULL,
  FOREIGN KEY (curso_id) REFERENCES curso(id),
  FOREIGN KEY (professor_id) REFERENCES professor(id)
);

-- Tabela Turma_Aluno (N:N)
CREATE TABLE turma_aluno (
  turma_id BIGINT NOT NULL,
```

```

    aluno_id BIGINT NOT NULL,
    PRIMARY KEY (turma_id, aluno_id),
    FOREIGN KEY (turma_id) REFERENCES turma(id),
    FOREIGN KEY (aluno_id) REFERENCES aluno(id)
);

-- Tabela Aula
CREATE TABLE aula (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    turma_id BIGINT NOT NULL,
    data_hora DATETIME NOT NULL,
    assunto VARCHAR(100) NOT NULL,
    FOREIGN KEY (turma_id) REFERENCES turma(id)
);

-- Tabela Nota
CREATE TABLE nota (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    aluno_id BIGINT NOT NULL,
    aula_id BIGINT NOT NULL,
    valor DOUBLE NOT NULL CHECK(valor >= 0 AND valor <= 10),
    FOREIGN KEY (aluno_id) REFERENCES aluno(id),
    FOREIGN KEY (aula_id) REFERENCES aula(id)
);

```

2. Exemplos de inserção de dados

```

-- Inserindo alunos
INSERT INTO aluno (nome, email, matricula, data_nascimento) VALUES
('Alice Silva', 'alice@email.com', '2025001', '2005-03-12'), ('Bruno Costa',
'bruno@email.com', '2025002', '2004-07-25');

-- Inserindo professores
INSERT INTO professor (nome, email, disciplina) VALUES
('Carlos Pereira', 'carlos@email.com', 'Matemática'),
('Daniela Souza', 'daniela@email.com', 'História');

-- Inserindo cursos
INSERT INTO curso (nome, codigo) VALUES
('Engenharia de Software', 'ENGSW01'),
('Design Gráfico', 'DG02');

-- Inserindo turmas
INSERT INTO turma (nome, curso_id, professor_id) VALUES
('Turma A', 1, 1),

```

```

('Turma B', 2, 2);
-- Matriculando alunos nas turmas
INSERT INTO turma_aluno (turma_id, aluno_id) VALUES
(1, 1),
(1, 2),
(2, 1);

-- Inserindo aulas
INSERT INTO aula (turma_id, data_hora, assunto) VALUES
(1, '2025-10-10 08:00:00', 'Introdução à Engenharia'),
(1, '2025-10-12 10:00:00', 'Programação Orientada a Objetos'),
(2, '2025-10-11 09:00:00', 'Fundamentos de Design');

-- Inserindo notas
INSERT INTO nota (aluno_id, aula_id, valor) VALUES
(1, 1, 8.5),
(2, 1, 7.0),
(1, 2, 9.0);

```

Entidade DTO Requisição DTO Resposta

Aluno nome, email, matricula, dataNascimento	dataNascimento
Professor nome, email, disciplina	id, nome, email, disciplina
Curso nome, codigo, listaProfessorIds	id, nome, codigo, listaProfessores (nomes)
Turma nome, cursold, professorId, listaAlunolds	id, nome, nomeCurso, nomeProfessor, listaAlunos (nomes)
Aula turmaId, dataHora, assunto id, nome, email, matricula,	id, nomeTurma, dataHora, assunto
Nota alunold, aulald, valor id, alunoNome, aulaAssunto, valor	

4. Endpoints sugeridos

Aluno

- **GET** /alunos → lista de AlunoDTO

- **GET /alunos/{id}** → AlunoDTO
- **POST /alunos** → AlunoDTO (req) → retorna AlunoDTO (resp) •
- PUT /alunos/{id}** → AlunoDTO (req) → retorna AlunoDTO (resp) •
- DELETE /alunos/{id}**

Professor, Curso, Turma, Aula, Nota

- **CRUD similar**
- **Endpoints extras:**
 - **GET /alunos/{id}/notas** → lista de NotaDTO
 - **GET /turmas/{id}/alunos** → lista de AlunoDTO
 - **GET /cursos/{id}/turmas** → lista de TurmaDTO

5. Atividades de fixação

1. Criar DTOs para cada entidade (Requisição e Resposta).
2. Criar Repositories JDBC usando PreparedStatement.
3. Criar Services que chamam repositories e convertem para DTOs.
4. Criar Controllers REST que usam apenas DTOs.
5. Implementar consultas JOIN para preencher DTOs com nomes/assuntos.
6. Validar dados antes de inserir (ex.: email único, nota entre 0 e 10).
7. Testar endpoints com POSTMAN ou outra ferramenta, garantindo que requisição e resposta respeitam os DTOs.